

Style guide: Ruby 2

Style guide: HTML 4

Style guide: Ruby

A [rubocop file](#) is provided in QPixel and rubocop is included in the QPixel bundle. Use `bundle exec rubocop` for Ruby style checking; you can also use `bundle exec rubocop -Da` for auto-correction.

In general, we follow “the” [Ruby Style Guide](#), with a few exceptions. The most important rules and those exceptions are detailed here. Exceptions and departures from the Style Guide are highlighted in **purple**.

Most editors can (and should!) be configured to automatically apply many of these code style rules for you, and to check your code as you write.

You should also read about [when it's appropriate to break these rules](#).

Indentation [↗](#)

Use spaces for indentation; do not use tab stops. Use two spaces per indentation level.

```
1 def my_method
2   'something'
3 end
```

Maximum Line Length [↗](#)

Limit lines to **120** characters. This rule is **not enforced for lines that must contain long URLs**, as splitting them generally introduces more chance for error and makes the code less readable. Use a linter exception to avoid generating rubocop errors for these.

Spaces & Braces [↗](#)

No space after `(` or `[` or before `)` or `]`. Use spaces around `{` and `}`, **including for hash literals**.

Spaces are **not** required around `{` and `}` when used for string interpolation.

```
1 # Not allowed:
2 some( arg )
3 [ 1, 2, 3 ]
4 ary.each{|e| puts e}
5 {a: 1, b: 2, c: 3}
6 "#{ interpolated }"
7
8 # Use instead:
9 some(arg)
10 [1, 2, 3]
11 ary.each { |e| puts e }
12 { a: 1, b: 2, c: 3 }
13 "#{interpolated}"
```

Line Continuation [↗](#)

Line continuation with `\` is only permitted for string literals. **Remember that long URLs should not be split and are exempt from maximum line lengths**.

Multi-line Method Chains [↗](#)

Use the leading `.` style and **align the continuation with the initial method call**:

```
1 # Not allowed:
2 one.two.three.
3     four
4
5 # Use instead:
6 one.two.three
7     .four
```

Style guide: HTML

All HTML contributions are expected to follow this style guide unless there's a good reason not to. Such exceptions should be documented in code comments and noted in pull requests.

Document [🔗](#)

HTML Version & Doctype [🔗](#)

Use HTML5. Always declare the correct doctype as the very first element in the document. Write doctype declarations in UPPERCASE.

```
1 <!DOCTYPE html>
```

Language [🔗](#)

Always declare the correct ISO 639-1 language and reading direction. Standardise on `en-US` for documents in English.

```
1 <html lang="en-US" dir="ltr"> ... </html>
```

Casing [🔗](#)

Write all tags and attributes in lowercase. Write attribute values in lowercase where possible.

Quotes [🔗](#)

Always use double quotes around attribute values. Use the HTML entity for quotes when the attribute value itself contains a double quote.

```
1 <link type="text/css" rel="stylesheet" href="https://codidact.org/assets/product.css" />
```

Void elements [🔗](#)

Always close all void elements. Leave a space between the last character of the tag and the trailing slash.

```
1 <!-- Not permitted: -->
2 <br/>
3 
4
5 <!-- Use instead: -->
6 <br />
7 
```

External resources [🔗](#)

When referencing external resources (including those local to the domain), always include the protocol. HTTPS is required for scripts and stylesheets, and is strongly recommended for all resources. Use canonical URIs (those that do not redirect upon request).

Indentation [🔗](#)

Use spaces - do not use tab stops. Use 2 spaces per level of indentation.

On element declarations whose attributes span more than one line, align subsequent lines with the first attribute on the first line:

```
1 <div id="question-select-modal" class="modal modal--urgent js-question-select"
```

```

2     title="Select your question here." data-merge-id="99182">
3     <p>Select a question.</p>
4 </div>

```

Where element content spans more than one line, start the content on the line after the tag and indent by one additional level:

```

1 <!-- Not permitted: -->
2 <p>Here is a paragraph with some really really really really really really really
3   really really really long content.</p>
4
5 <!-- Use instead: -->
6 <p>
7   Here is a paragraph with some really really really really really really really
8   really really really long content.
9 </p>

```

Semantic elements [↗](#)

Use semantic elements where appropriate: use `<main>` over `<div class="main">`, or `<footer>` over `<div class="footer">`, etc.

Line length [↗](#)

Limit lines to 120 characters. This rule is not enforced where lines contain long URLs.

Head [↗](#)

Encoding [↗](#)

Use UTF-8 encoding and ensure your editor is set to use UTF-8 encoding. Always declare the document encoding as the first element in the `<head>` section.

```

1 <meta charset="utf-8" />

```

Viewport [↗](#)

Correctly declare the viewport. Applications SHOULD be responsive, but if they are not, do not declare the viewport as mobile-friendly. Generally, do not set `user-scalable=no`.

```

1 <!-- For responsive applications: -->
2 <meta name="viewport" content="width=device-width, initial-scale=1" />
3 <!-- For non-responsive applications (adjust width as necessary): -->
4 <meta name="viewport" content="width=1024" />

```

Title [↗](#)

Ensure all pages have a title. Include the *page* title first; if a community or network name is included place it at the end.

```

1 <title>Questions | Codidact</title>

```

Description [↗](#)

Ensure all pages have a declared meta description, unless the page is not intended for public consumption.

```

1 <meta name="description" content="Questions on Codidact, a Q&A community." />

```

Links & scripts [↗](#)

Add links in this order:

- Preloads/prefetches
- Favicon
- `rel="canonical"`
- Any other links
- CSS assets

Scripts should be loaded `async` or `defer` where possible, or included at the end of the `<body>` instead.

Fonts [↗](#)

Consider using a system-native font stack from CSS:

```
1 body {
2   font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Roboto', 'Helvetica',
3     'Arial', sans-serif, 'Apple Color Emoji', 'Segoe UI Emoji', 'Segoe UI Symbol';
4 }
```

Where this is decided against, non-standard fonts should be added via CSS and SHOULD be hosted locally - i.e. no third-party requests for fonts. This is due to privacy considerations. Always include `local()` in the `@font-face` definition so that users who have the font installed can avoid the resource load. Always include appropriate fallback fonts and a generic font type fallback.

```
1 @font-face {
2   font-family: 'Source Code Pro';
3   font-style: normal;
4   font-weight: 400;
5   font-display: swap;
6   src: local('Source Code Pro Regular'), local('SourceCodePro-Regular'),
7     url('/assets/fonts/HI_SiYsKILxRpg3hIP6sJ7fM7Pq1PevW.woff2') format('woff2');
8   unicode-range: U+0000-00FF, U+0131, U+0152-0153, U+02BB-02BC, U+02C6, U+02DA,
9     U+02DC, U+2000-206F, U+2074, U+20AC, U+2122, U+2191, U+2193, U+2212, U+2215,
10    U+FEFF, U+FFFD;
11 }
12
13 .code-block {
14   font-family: 'Source Code Pro', 'Consolas', 'Lucida Console', monospace;
15 }
```

⚠ Non-standard fonts

Remember that the list of fonts which are reliably available cross-platform is very small. Arial and a handful of others are about the only fonts you can rely on to be available. Either use a system font stack instead, or load web fonts.

Body [↗](#)

JavaScript references [↗](#)

When adding an ID or class to reference an element from JavaScript, prefix the value with `js-`.

```
1 <div class="modal modal--urgent js-question-select"> ... </div>
```

Links (`<a>`) [↗](#)

- Generally, avoid using `target="_blank"` as it deprives users of the choice of where to open a link.
- If you do use `target="_blank"` to open links in a new tab, also include `rel="noopener noreferrer"`.

JS-enabled links [↗](#)

As a general rule, avoid `href="#"` and `href="javascript:void(0)"` links:

- If a JS-enabled link provides functionality in-page for which there also exists a dedicated page, use `<a>` with a `href` pointing to that dedicated page, then call `event.preventDefault()` from JS to avoid navigation and handle the functionality in-page.
- If there is no dedicated page for the functionality provided, use a `<button>` (and style it as a link if necessary). Ideally, inject the button from JS to avoid users who can't use the button seeing it - unless this would cause more confusion by its absence.

Images [↗](#)

- Use a compressed image format or small file size where possible. Resize images for the size at which they will be used.
- Make use of `<picture>` / `<srcset>` where possible.
- Load images asynchronously where possible.

Headings [↗](#)

- Ensure all pages have a level 1 header (`<h1>`) that is not the website name.
- Ensure pages do not have more than one `<h1>` .
- Use headings in order; style via CSS rather than using a smaller heading level.

Inputs [↗](#)

- Use specific input types such as `type="email"` rather than `type="text"` for every field.
- Ensure every input has a unique ID and a `<label>` with a `for` attribute that points to it.
- Make use of `aria` attributes including `aria-label` .
- Make use of `inputmode` to tell mobile devices which keyboard to show.

Keyboard navigation [↗](#)

Ensure all pages are navigable using the keyboard only, and all interactive elements and inputs are reachable and usable.

Progressive enhancement [↗](#)

Make sure major features are usable without JavaScript enabled. Enhance basic functionality from JS when it is available.